

```
public class Cars {
    int price;
    int speed;
    String color;

    public Cars() {
        // TODO Auto-generated constructor stub
    }
    public Cars(int price, int speed, String color) {
        // TODO Auto-generated constructor stub
        this.price = price;
        this.speed = speed;
        this.color = color;
    }
}

public class Cars_Client {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        Cars[] arr = new Cars[5];
        arr[0] = new Cars(200, 10, "White");// P S C 2x
        arr[1] = new Cars(1000, 20, "Black");// - 3x
        arr[2] = new Cars(345, 3, "Yellow");
        arr[3] = new Cars(34, 89, "Grey");
        arr[4] = new Cars(8907, 6, "Red");
    }
}
```

cars

```
public static <T>void Sort(T[] arr) {
    for (int turn = 1; turn < arr.length; turn++) {
        for (int i = 0; i < arr.length - turn; i++) {
            if (arr[i] > arr[i + 1]) {
                T temp = arr[i];
                arr[i] = arr[i + 1];
                arr[i + 1] = temp;
            }
        }
    }
}
```

2x	3x	4x	6x	8x
0	1	2	3	4

2x > 3x
2x > 4x
2x > 6x
2x > 8x

3x > 4x
3x > 6x
3x > 8x

4x > 6x
4x > 8x

6x > 8x

8x > 5x
8x > 0
8x < 5x
8x == 5x

this.p - other.p > 0 | 0. See this file

```
public static <T extends Comparable<T>> void Sort(T[] arr) {
    for (int turn = 1; turn < arr.length; turn++) {
        for (int i = 0; i < arr.length - turn; i++) {
            if (arr[i].compareTo(arr[i + 1]) > 0) {
                T temp = arr[i];
                arr[i] = arr[i + 1];
                arr[i + 1] = temp;
            }
        }
    }
}
```

cars[] arr = new Cars[5];
arr[0] = new Cars(200, 10, "White");// P S C
arr[1] = new Cars(1000, 20, "Black");
arr[2] = new Cars(345, 3, "Yellow");
arr[3] = new Cars(34, 89, "Grey");
arr[4] = new Cars(8907, 6, "Red");

this.p = 200
0.p = 1000

this.p - other.p
7 - 5

arr[0].compareTo(arr[1]) > 0

Let's say you have a big schedule of activities. Your aim is to do as much as activities as possible.

TIME →

3
9
5

6
7 9
0 10
4 5
8 9
4 10
5 7

4 5
5 7
8 9
7 9
4 10
6 10

4 5
5 7
7 9 → 8 9

root.right = null
root.right = root.left
root.left = null

tail

```
public TreeNode MakeLL(TreeNode root) {
    if (root == null) {
        return null;
    }

    TreeNode left_tail = MakeLL(root.left);
    TreeNode right_tail = MakeLL(root.right);
    if (left_tail != null) {
        left_tail.right = root.right; // head right wala ka
        root.right = root.left;
        root.left = null;
    }
}
```